Virtuális egér — a MediaPipe-os megoldás algoritmusa

Implementációs leírás · Virtual Mouse csapat, 2026

Bevezető

A feladat: webkamerából egérvezérlés. A kamera képéből felismerjük a kezet, a mutatóujj pozíciójából kurzormozgást csinálunk, az ujjak összecsípéséből kattintást.

A kézfelismerést a Google MediaPipe Hands végzi — ez egy előtanított modell, ami kb. 30 000 kézfelvételen lett betanítva, nekünk ehhez nem kell adatot gyűjteni vagy modellt tanítani. A MediaPipe kiadja a kéz 21 referenciapontjának koordinátáit. Innentől a mi dolgunk: ezekből a koordinátákból kurzorpozíciót, kattintást, húzást számolni.

Pipeline



Az első négy lépést a MediaPipe intézi. Az utolsó blokk a mi kódunk.

A 21 referenciapont

A MediaPipe minden frame-en visszaad 21 pontot. Minden ponthoz tartozik egy `x`, `y` (normalizált, 0–1, a kép szélességéhez/magasságához képest) és egy `z` (relatív mélység, a csuklóhoz képest). Nekünk nem mind a 21 pont kell, hanem:

Pont	Pozíció	Használat
`4`	Hüvelykujj hegye	Kattintás detektálása (4–8 távolság)
`5`	Mutatóujj töve	Normalizáláshoz (tenyérméret)
`8`	Mutatóujj hegye	Kurzorpozíció
`12`	Középsőujj hegye	Görgetés gesztus
`17`	Kisujj töve	Normalizáláshoz (tenyérméret)

Az algoritmus

1. Frame beolvasása és kézdetektálás

OpenCV-vel olvassuk a kamerát. A MediaPipe RGB-t vár, az OpenCV BGR-t ad, úgyhogy konvertálni kell.

```
frame = camera.read()
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
result = hands.process(frame_rgb)

if not result.multi_hand_landmarks:
    continue  # no hand detected, skip frame

landmarks = result.multi_hand_landmarks[0].landmark
```

A MediaPipe-ot `max_num_hands=1` és `min_detection_confidence=0.7` beállítással használjuk.

2. Kurzorpozíció kiszámítása

A 8. pont (mutatóujj hegye) adja a kurzor helyét. A normalizált koordinátákat le kell képezni a képernyő felbontására.

Ha egyszerűen szoroznánk (`x * screen_width`), a felhasználónak a kézt a kamera legszéléig kellene vinnie ahhoz, hogy a kurzor a képernyő szélére érjen. Ehelyett egy szűkebb tartományból dolgozunk, és azt nyújtjuk a teljes képernyőre:

```
# Map the camera's usable region [0.2, 0.8] to the full screen
raw_x = np.interp(landmarks[8].x, [0.2, 0.8], [0, screen_w])
raw_y = np.interp(landmarks[8].y, [0.2, 0.8], [0, screen_h])
```

A `0.2` és `0.8` értékek hangolhatóak. Szűkebb tartomány = érzékenyebb mozgás, tágabb = nagyobb kézmozgás kell.

3. Simítás (exponenciális mozgóátlag)

A nyers pozíció remeg, mert a modell becslése frame-ről frame-re ugrál pár pixelt. Ezt exponenciális mozgóátlaggal szűrjük:

$$S_t = \alpha \cdot P_t + (1 - \alpha) \cdot S_{t-1}$$

Ahol P_t a nyers pozíció, S_t a simított (amit ténylegesen a kurzornak beállítunk), és α a simítási faktor.

```
# alpha between 0.2 (very smooth, sluggish) and 0.5 (responsive, jittery)
ALPHA = 0.3

smooth_x = ALPHA * raw_x + (1 - ALPHA) * prev_x
smooth_y = ALPHA * raw_y + (1 - ALPHA) * prev_y

pyautogui.moveTo(smooth_x, smooth_y)

prev_x = smooth_x
prev_y = smooth_y
```

4. Kattintás — csippentés gesztus, hiszterézissel

A kattintást a hüvelykujj hegye (4) és a mutatóujj hegye (8) közötti távolságból számoljuk. A nyers euklideszi távolság azonban függ attól, milyen messze van a kéz a kamerától. Ezért normalizáljuk: elosztjuk a tenyér szélességével (5–17 pontok távolsága), ami aránylag állandó.

```
def distance(a, b):
    return math.sqrt((a.x - b.x)**2 + (a.y - b.y)**2)

d_fingers = distance(landmarks[4], landmarks[8])
d_palm    = distance(landmarks[5], landmarks[17])
d_norm    = d_fingers / d_palm
```

Egyetlen küszöbbel az lenne a baj, hogy a távolság ingadozása a küszöb körül gyors kattintás-sorozatot okozna. Ezért két küszöböt használunk (hiszterézis):

```
d_norm:  0.0 ──────── 0.20 ────── 0.30 ──────── 1.0
         [  PRESSED  ] [ dead ] [ RELEASED    ]
                         zone
```

```
PRESS_THRESHOLD   = 0.20
RELEASE_THRESHOLD = 0.30

# state persists between frames, starts as "released"

if d_norm < PRESS_THRESHOLD and state == "released":
    state = "pressed"
    pyautogui.mouseDown()

if d_norm > RELEASE_THRESHOLD and state == "pressed":
    state = "released"
    pyautogui.mouseUp()
```

A `0.20` és `0.30` közötti sáv a holtsáv: itt nem történik semmi, az állapot marad ami volt. Ez akadályozza meg a vibrálást.

5. Húzás (drag)

Ezt nem kell külön kezelni. Ha az állapot `"pressed"` és közben a kurzor mozog, az OS szintjén automatikusan drag lesz belőle, mert az egérgomb le van nyomva miközben a kurzor pozíciója változik.

6. Görgetés (scroll)

Ha a mutatóujj és a középsőujj is ki van nyújtva (a hüvelyk nem), és együtt mozognak felfelé/lefelé:

```
# detect two-finger-up gesture (index + middle extended, thumb folded)
# then track vertical movement between frames

if is_scroll_gesture(landmarks):
    dy = prev_scroll_y - landmarks[8].y
    if abs(dy) > SCROLL_THRESHOLD:
        pyautogui.scroll(int(dy * SCROLL_SPEED))
```

Hogy egy ujj ki van-e nyújtva, az ujjhegy (pl. 8) és az ujjtő (pl. 5) y-koordinátájának összehasonlításából dönthető el.

Teljes főciklus

```
camera = cv2.VideoCapture(0)
hands = mp.solutions.hands.Hands(
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)

ALPHA = 0.3
PRESS_THRESHOLD = 0.20
RELEASE_THRESHOLD = 0.30
FRAME_L, FRAME_R = 0.2, 0.8  # usable camera region

state = "released"
prev_x, prev_y = 0.0, 0.0
screen_w, screen_h = pyautogui.size()

while True:
```

```
    ret, frame = camera.read()
    if not ret:
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = hands.process(frame_rgb)

    if not result.multi_hand_landmarks:
        continue

    lm = result.multi_hand_landmarks[0].landmark

    # cursor position
    raw_x = np.interp(lm[8].x, [FRAME_L, FRAME_R], [0, screen_w])
    raw_y = np.interp(lm[8].y, [FRAME_L, FRAME_R], [0, screen_h])

    # smoothing
    smooth_x = ALPHA * raw_x + (1 - ALPHA) * prev_x
    smooth_y = ALPHA * raw_y + (1 - ALPHA) * prev_y
    prev_x, prev_y = smooth_x, smooth_y

    pyautogui.moveTo(smooth_x, smooth_y)

    # click detection
    d_fingers = distance(lm[4], lm[8])
    d_palm = distance(lm[5], lm[17])
    d_norm = d_fingers / d_palm

    if d_norm < PRESS_THRESHOLD and state == "released":
        state = "pressed"
        pyautogui.mouseDown()

    if d_norm > RELEASE_THRESHOLD and state == "pressed":
        state = "released"
        pyautogui.mouseUp()

camera.release()
```

Függőségek

```
pip install mediapipe opencv-python numpy pyautogui
```

> Wayland alatt (Hyprland, Sway stb.) a PyAutoGUI nem működik, mert X11-re épül. Helyette: `ydotool` vagy a `wayland-automation` Python csomag.

Hangolható paraméterek

Paraméter	Alapérték	Hatás
`ALPHA`	0.3	Kisebb → simább de lomhább kurzor. Nagyobb → gyorsabb reakció de remegős.
`PRESS_THRESHOLD`	0.20	Kisebb → szorosabban kell csípni a kattintáshoz.
`RELEASE_THRESHOLD`	0.30	Nagyobb → messzebbre kell húzni az ujjakat az elengedéshez.
`FRAME_L`, `FRAME_R`	0.2, 0.8	A kamerakép mekkora részét használjuk. Szűkebb → érzékenyebb.
`min_detection_confidence`	0.7	Kisebb → több false positive. Nagyobb → néha nem detektálja a kezet.

Miért a MediaPipe?

A kiírás a CONDENSATION algoritmust ajánlotta. Mi is átnéztük, plusz a CNN/YOLO vonalat. Az összehasonlítás:

	CONDENSATION	CNN / YOLO	MediaPipe
Tanítóadat	Nem kell	Kell, sok	Nem kell
Implementáció	Nehéz (4–8 hét)	Közepes (napok–hetek)	Egyszerű (pár óra)
Kurzor pozíció	Kontúrból becsült	Bounding box közepéből	Közvetlen (8. pont)
Kattintás	Saját szabályt kell írni	Külön osztályt kell tanítani	Két pont távolsága
Sebesség	CPU-n megy	GPU kell a tanításhoz	CPU-n valós idejű

A lényeg: a MediaPipe-pal az időnk nagy részét az egérvezérlés logikájára (simítás, gesztusok, felhasználói élmény) tudjuk fordítani, nem a kézdetektálás megírására. A szűrős (CONDENSATION-szerű) gondolat pedig megmarad a simításban, csak részecskék nélkül.

`Virtual Mouse csapat · Ilonczai, Soczó, Kormányos, Somogyi · Szeged, 2026`